Comment l'optimisation des dissipateurs thermiques peut-elle améliorer l'efficacité énergétique des composants électroniques ?

ABDUL OMAR
N° SCEI : 21733

Introduction

- Lorsqu'un appareil électronique fonctionne, ses composants consomment de **l'énergie**.
- Une partie de cette énergie est transformée en **chaleur**.
- Cette chaleur s'accumule et peut provoquer :
  - Une **baisse des performances**,
  - Des **pannes irréversibles**.
- Il est donc nécessaire de **refroidir** ces composants pour garantir leur **bon fonctionnement**.

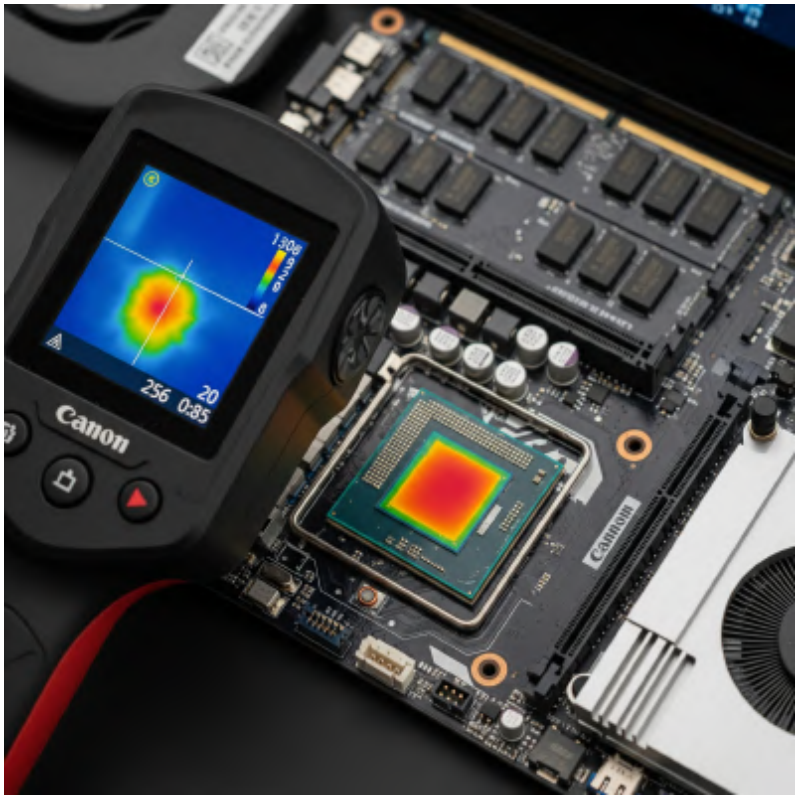


Figure: Température d'un processeur [1]

Introduction

- **Fonctionnement d'un dissipateur** :
  - **Conduction** : transfert de chaleur du composant vers le dissipateur.
  - **Convection** : dissipation de la chaleur dans l'air ambiant.

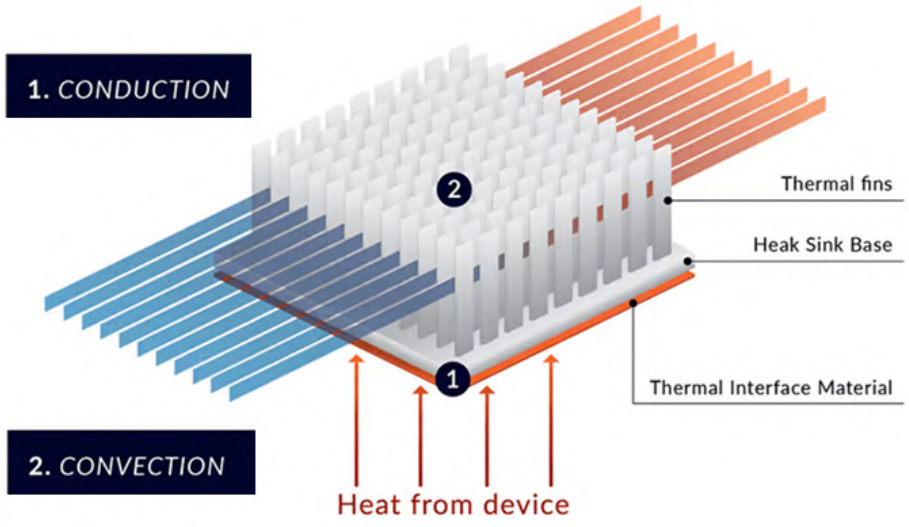


Figure: Fonctionnement d'un dissipateur thermique [2]

Objectifs

- Maintenir les composants électroniques à **des températures sûres**.
- Prolonger leur **durée de vie**.
- Garantir des **performances optimales**.

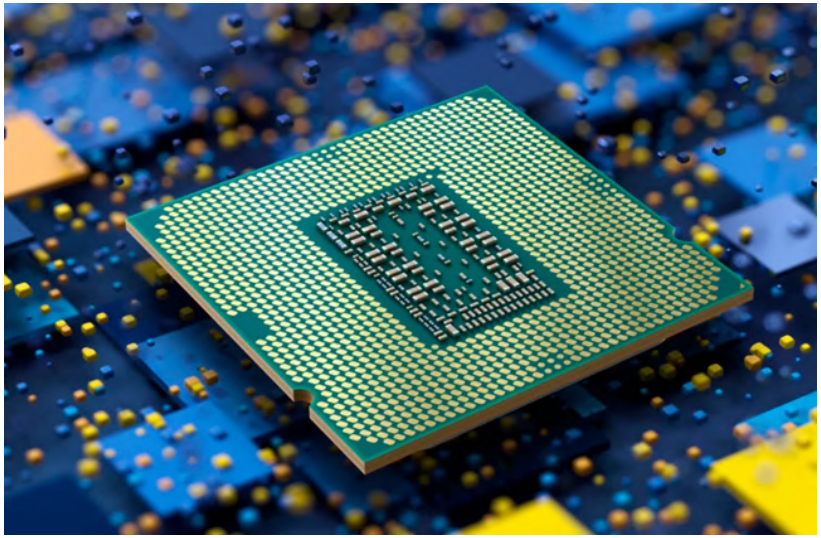


Figure: Processeur [3]

Théorie

Loi de Fourier

$$\boxed{\vec{j_{th}} = -\lambda \cdot \vec{\nabla} T}$$

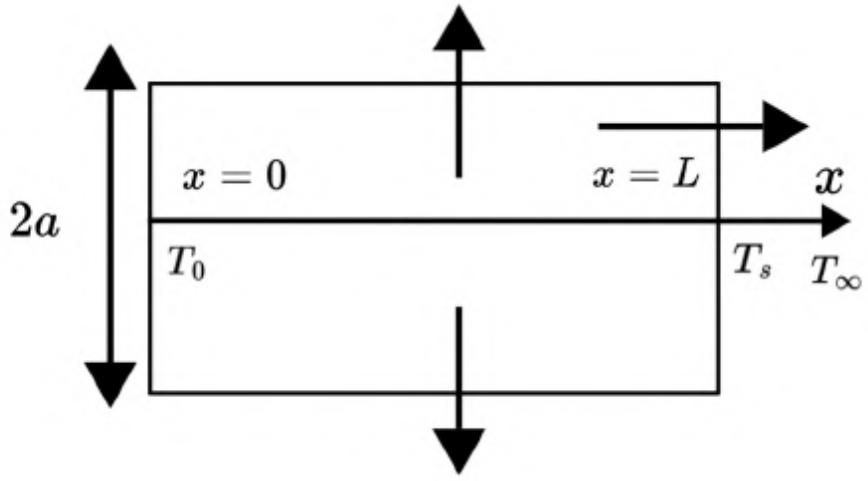
- $\vec{j_{th}}$: Densité de courant thermique $W \cdot m^{-2}$
- λ : Conductivité thermique $W \cdot m^{-1} \cdot K^{-1}$
- T : Température K

Loi de Newton

$$\boxed{\varphi_{\text{fluide}\to\text{solide}} = h \cdot S \cdot (T_{\text{fluide}} - T_{\text{solide}})}$$

- $\varphi_{\text{fluide}\to\text{solide}}$: Flux thermique W
- h : Coefficient conducto-convectif $W \cdot m^{-2} \cdot K^{-1}$
- S : Surface d'échange m^2
- $T_{\text{fluide}}, T_{\text{solide}}$: Température fluide/solide K

Théorie

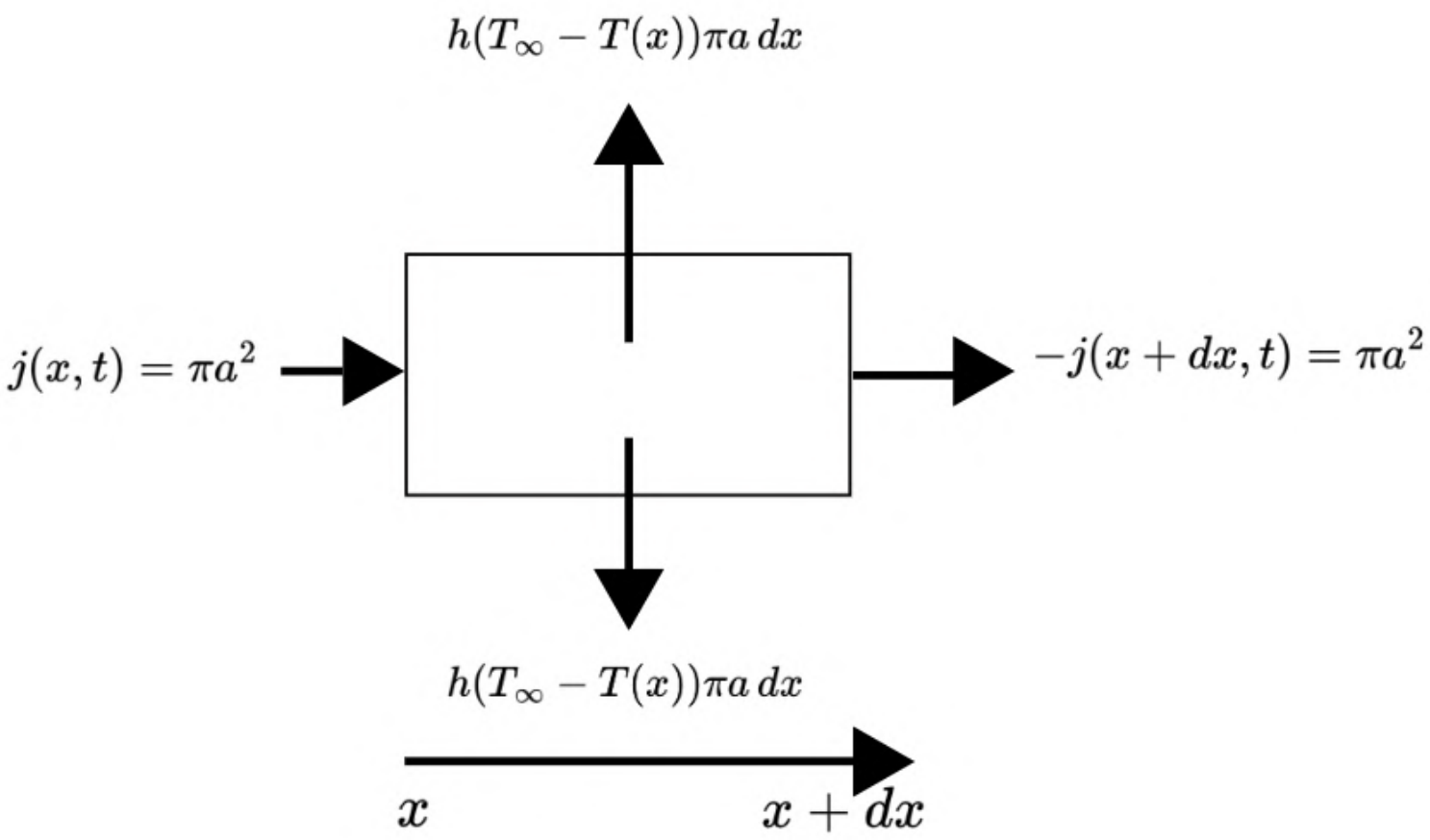


Ailette: rayon a et de longueur L

- T_0 : Température de la surface située en $x = 0$ en Kelvin
- T_s : Température à l'extrémité de l'ailette en $x = L$ en Kelvin
- T_∞ : Température ambiante en Kelvin

Théorie

Premier principe de la thermodynamique appliqué à une tranche: transformation isobare entre t et $t + dt$



Théorie

$$\delta Q = j_Q(x) \times \pi a^2 \mathrm{d}t - j_Q(x + \mathrm{d}x) \times \pi a^2 \mathrm{d}t + h(T_\infty - T(x)) \times 2\pi a \mathrm{d}x \times \mathrm{d}t$$

$$\delta Q = \rho \cdot c \cdot \pi a^2 \, dx \cdot \frac{\partial T}{\partial t} \, dt$$

Dans la limite du calcul différentiel

$$\boxed{\frac{\rho c}{\lambda} \frac{\partial T}{\partial t} = \frac{\partial^2 T}{\partial x^2} + \frac{2h}{\lambda a} (T_\infty - T)}$$

Théorie

Compromis

- Dimensionnement des ailettes : dissipateur
  $20\,\text{mm} \times 20\,\text{mm} \times 20\,\text{mm}$
- Choix du matériau pour une bonne conductivité thermique
- Optimisation de la surface d'échange thermique pour maximiser la dissipation
- Adaptation à l'espace disponible dans l'ordinateur
- Coût faible
- Durabilité
- Résistance à la corrosion

Théorie

Propriétés	**Aluminium**	**Cuivre**
Conductivité thermique élevée	✓	✓
Densité faible	✓	
Coût faible	✓	
Résistance à la corrosion	✓	
Conductivité électrique élevée		✓
Capacité thermique élevée		✓
Facilité d'usinage	✓	

Tableau : Comparatif Aluminium / Cuivre
Source : Données issues de diverses publications techniques sur les matériaux thermiques, notamment "Thermal Conductivity of Metals" – MatWeb.

Manipulation

Pourquoi utiliser une LED de puissance plutôt qu'un processeur ?

- **Reproductibilité** : Conditions de chauffe plus stables et contrôlables.
- **Bonne analogie thermique** : La LED de puissance, comme un processeur, dissipe la chaleur dans un petit volume.

Manipulation

Caractéristiques de la LED :

- Intensité : **400 mA**
- Tension : **9–12 V**
- Puissance : **10 W**

Cette LED de puissance nécessite impérativement un **dissipateur thermique** pour éviter la surchauffe.

Figure: LED de puissance [4]

Manipulation

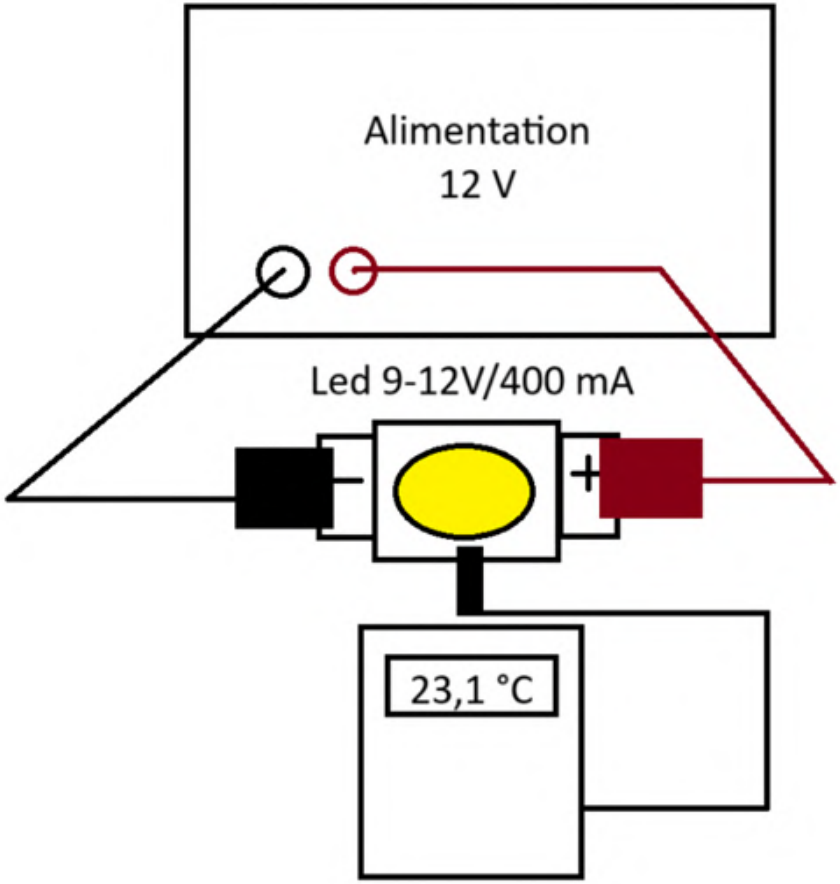
Matériel

- Dissipateur en aluminium
- LED de puissance
- Câbles et pinces crocodiles
- Thermomètre
- Alimentation 12 V

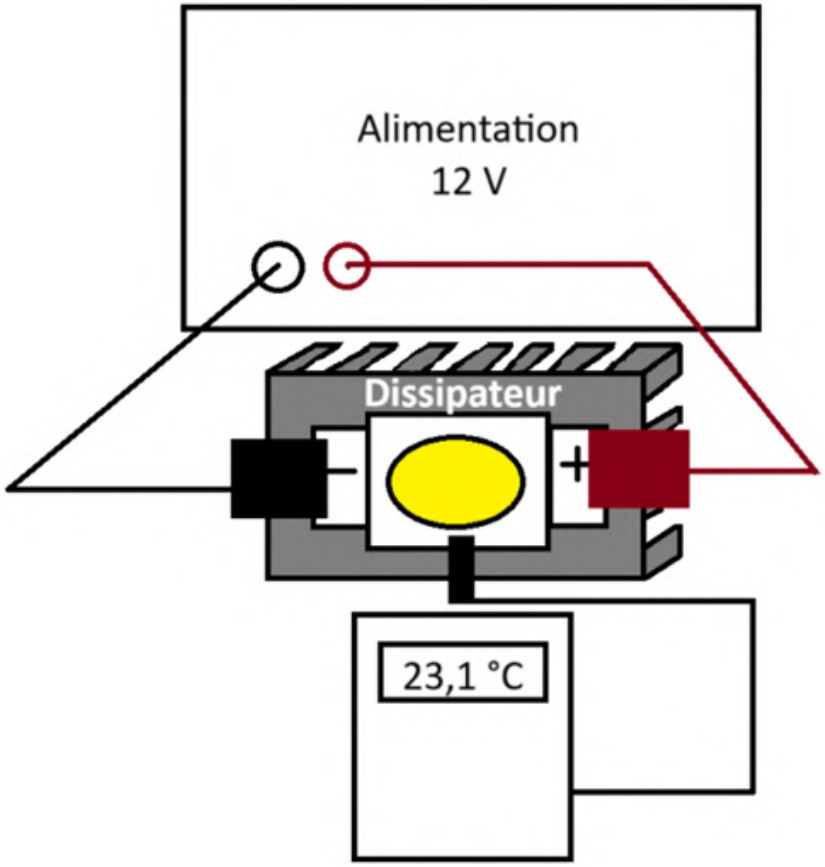
Protocole

- Brancher la LED à l'alimentation avec les pinces crocodiles
- Relever la température de la LED toutes les 5 minutes
- Installer le dissipateur
- Mesurer toutes les 5 minutes la température de la LED, de la base du dissipateur et de l'extrémité d'une ailette
- Effectuer plusieurs séries de mesures et comparer les résultats

Manipulation



Sans dissipateur



Avec dissipateur

Manipulation

Figure: Thermomètre utilisé

Figure: Thermomètre Checktemp [5]

Manipulation

Précision	± 0.2°C (-20 à 90°C) ± 0.3°C (en dehors)
Résolution T° (°C)	± 0.1

Figure: Incertitudes [5][6]

Incertitudes du thermomètre :

- **Précision** : $\pm$ 0.2 °C (Donnée constructeur)
- **Résolution** : $\pm$ 0.1 °C (Donnée constructeur)
- **Positionnement** : Placement de la sonde (Incertitude Type A)

Manipulation

Pour chaque valeur moyenne de température (moyenne sur 10 mesures), on calcule l'incertitude de type A :

$$u_A = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}\left(T_i - \overline{T}\right)^2} \quad \text{avec } N = 10$$

L'incertitude globale combinée est obtenue par la somme quadratique des incertitudes **indépendantes** :

$$\boxed{u = \sqrt{u_A^2 + u_{\text{précision}}^2 + u_{\text{résolution}}^2}}$$

$$u_{\text{précision}} = 0.2\,^\circ C, \quad u_{\text{résolution}} = 0.1\,^\circ C$$

Manipulation

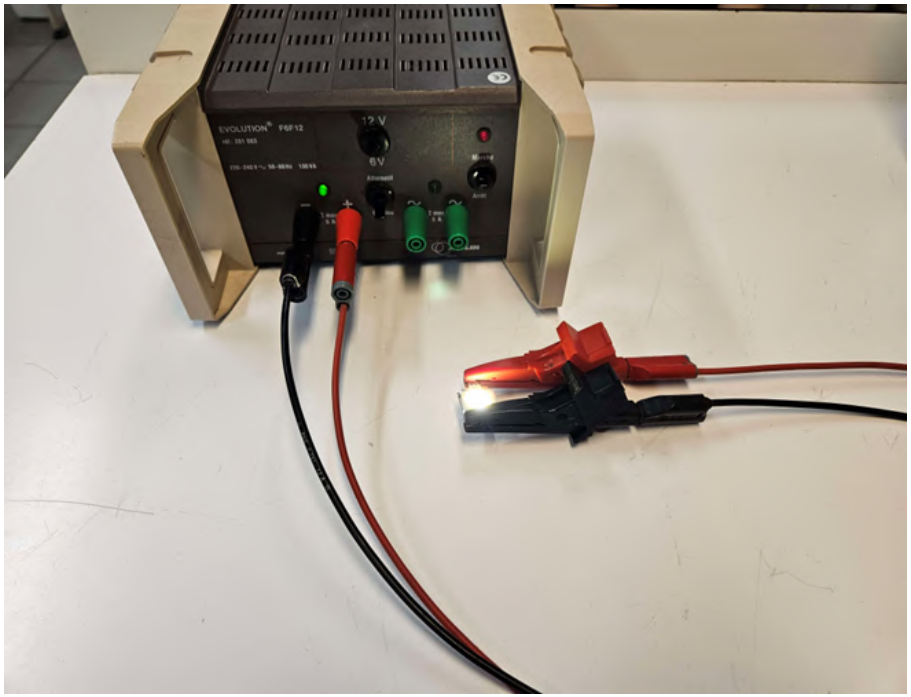
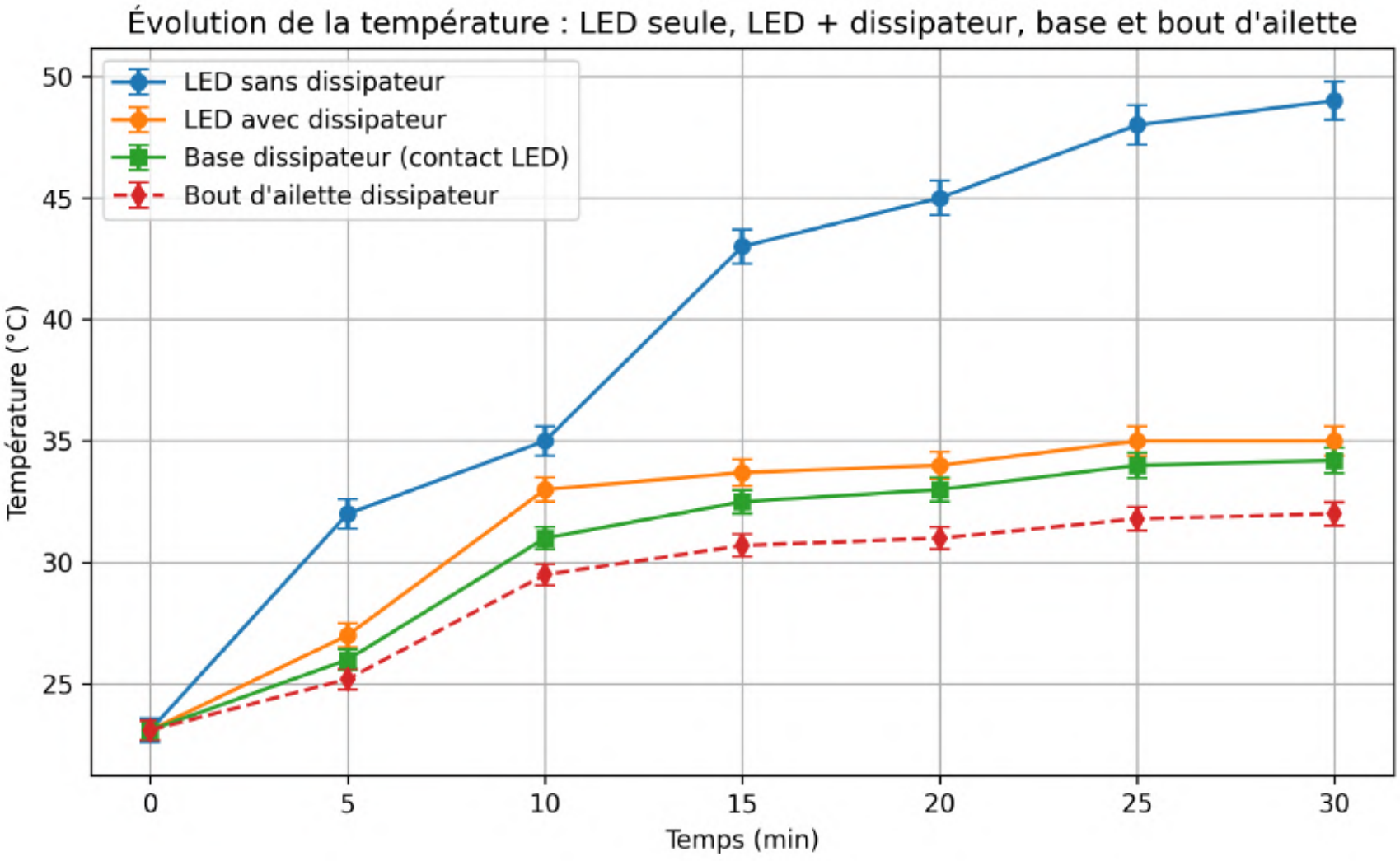


Figure: Sans dissipateur

Figure: Dissipateur fixé à la LED avec un pad thermique

Manipulation



Simulation

- **Modélisation** de la LED.
- **Analyse thermique transitoire**.
- **Modélisation du dissipateur** (Aluminium, dimensions réelles).
- **Assemblage** Σ = LED + dissipateur.
- **Transferts thermiques** :
  - Conduction,
  - Convection,
  - Échauffement interne.
- **Observation de la température Σ en fonction du temps**.

Figure: Modélisation FreeCAD [7]

Simulation

Calcul du flux thermique de surface :
Hypothèse : puissance uniformément répartie

$$\boxed{\phi = \frac{P}{S} = \frac{10}{1 \times 10^{-4}} = 100\,000\,\mathrm{W} \cdot \mathrm{m}^{-2}}$$

FreeCAD : Flux thermique appliqué sur la face supérieure de la LED.

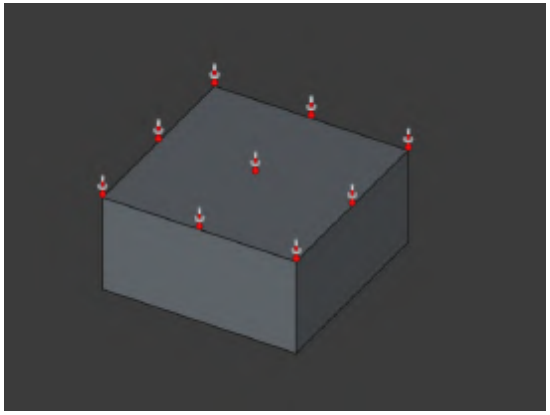
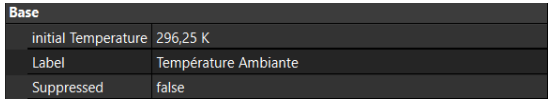


Figure: Dimensions LED
10 mm × 10 mm × 5 mm

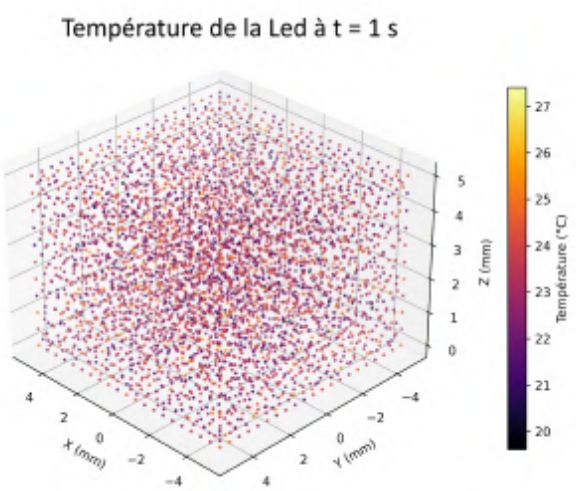


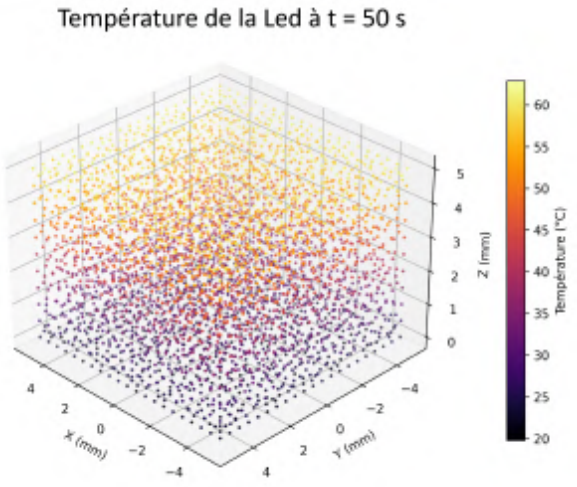
Base	
initial Temperature	296,25 K
Label	Température Ambiante
Suppressed	false

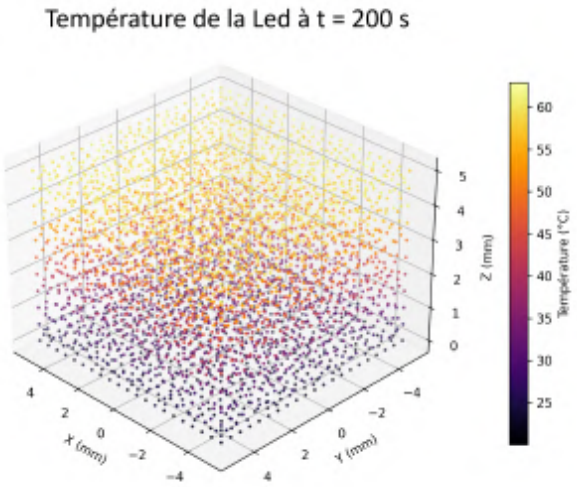
Figure: Température ambiante 23.1 °C

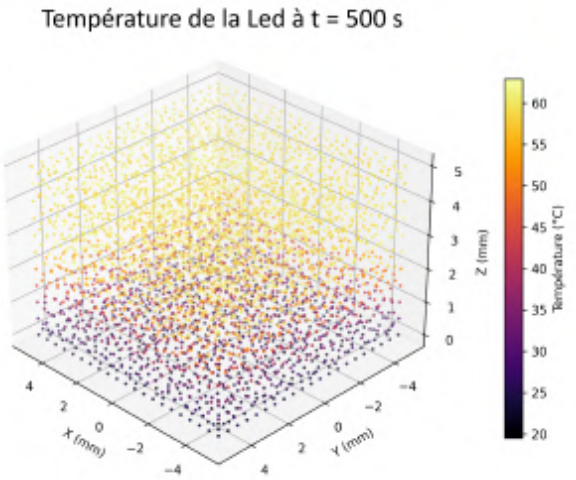
Figure: Flux de chaleur appliqué $100\,000\,\mathrm{W} \cdot \mathrm{m}^{-2}$

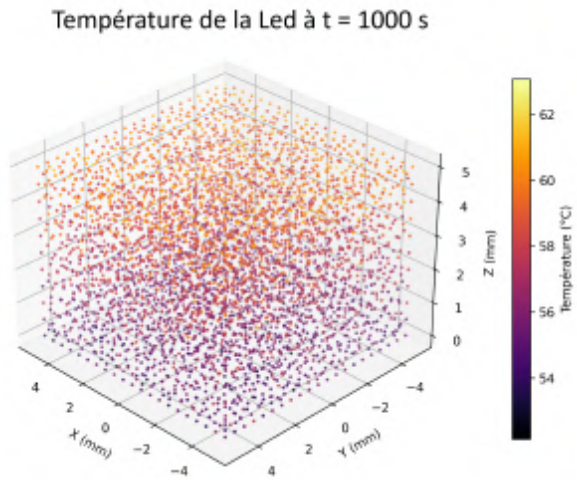
Simulation

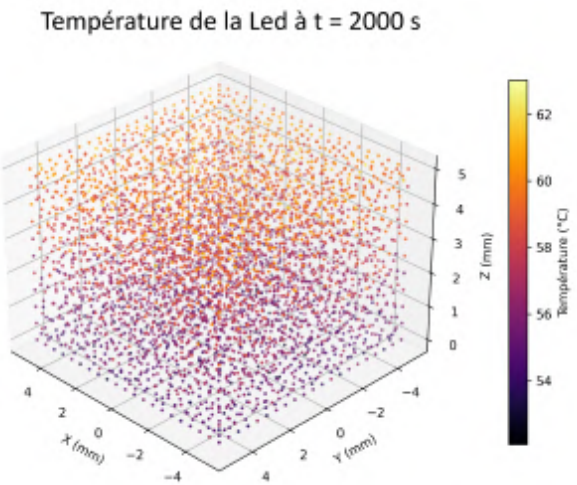

Température de la Led à t = 1 s
X (mm)
Y (mm)
Z (mm)
Température (°C)


Température de la Led à t = 50 s
X (mm)
Y (mm)
Z (mm)
Température (°C)


Température de la Led à t = 200 s
X (mm)
Y (mm)
Z (mm)
Température (°C)


Température de la Led à t = 500 s
X (mm)
Y (mm)
Z (mm)
Température (°C)


Température de la Led à t = 1000 s
X (mm)
Y (mm)
Z (mm)
Température (°C)


Température de la Led à t = 2000 s
X (mm)
Y (mm)
Z (mm)
Température (°C)

Simulation

- **Dissipateur** :
  - 100 ailettes
  - Dimensions : 20 mm $\times$ 20 mm $\times$ 20 mm
- **LED** :
  - Assemblée sur la base du dissipateur pour une meilleure dissipation de chaleur.

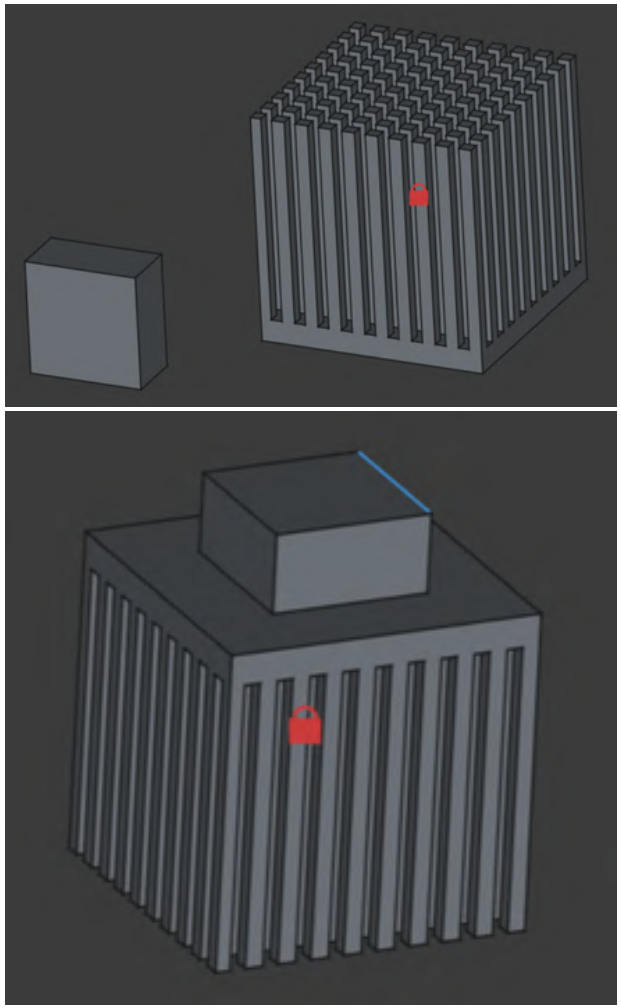


Figure: Dissipateur + LED

Simulation

- **Application des matériaux** :
  - Dissipateur : **Aluminium**
  - LED : **GaN / InGaN**
- **Maillage** :
  - Application du maillage Gmsh de FreeCAD à l'assemblage

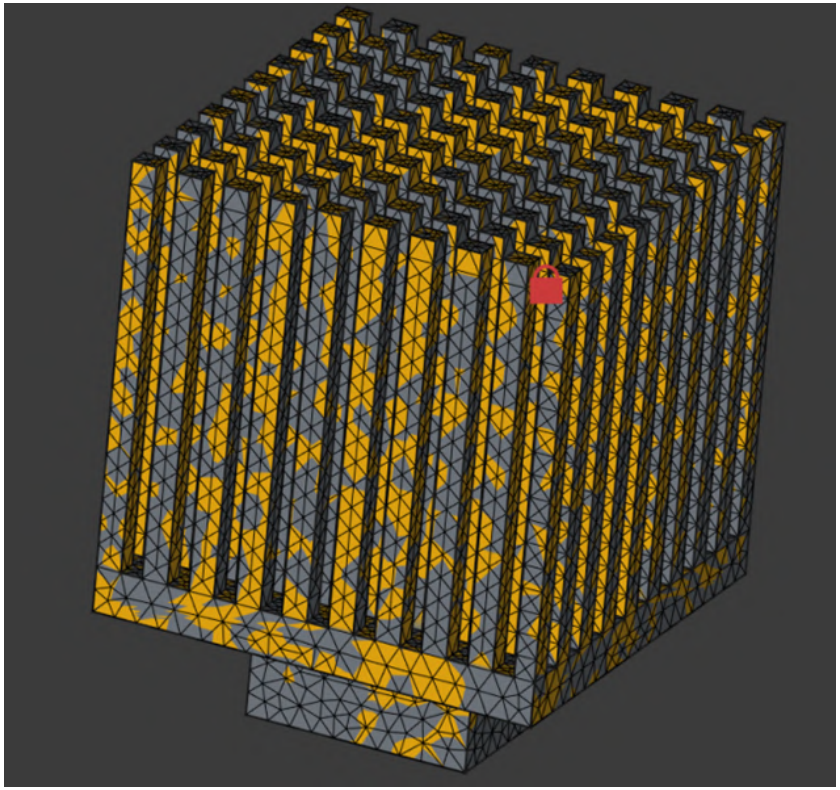


Figure: Application du maillage à l'assemblage

Simulation

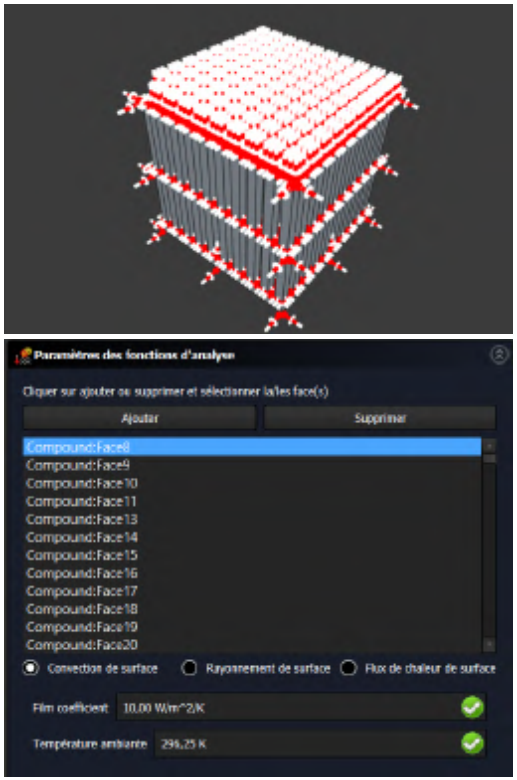


Figure: Application de la convection sur les ailettes du dissipateur

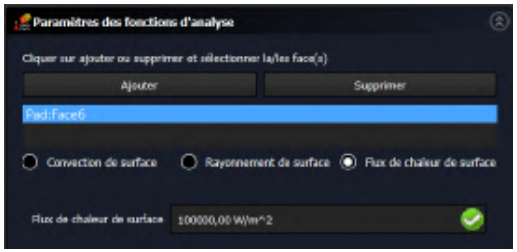


Figure: Flux de chaleur $100\,000\,\mathrm{W \cdot m^{-2}}$ appliqué sur la surface de contact LED/Dissipateur

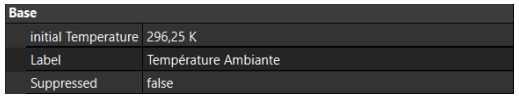
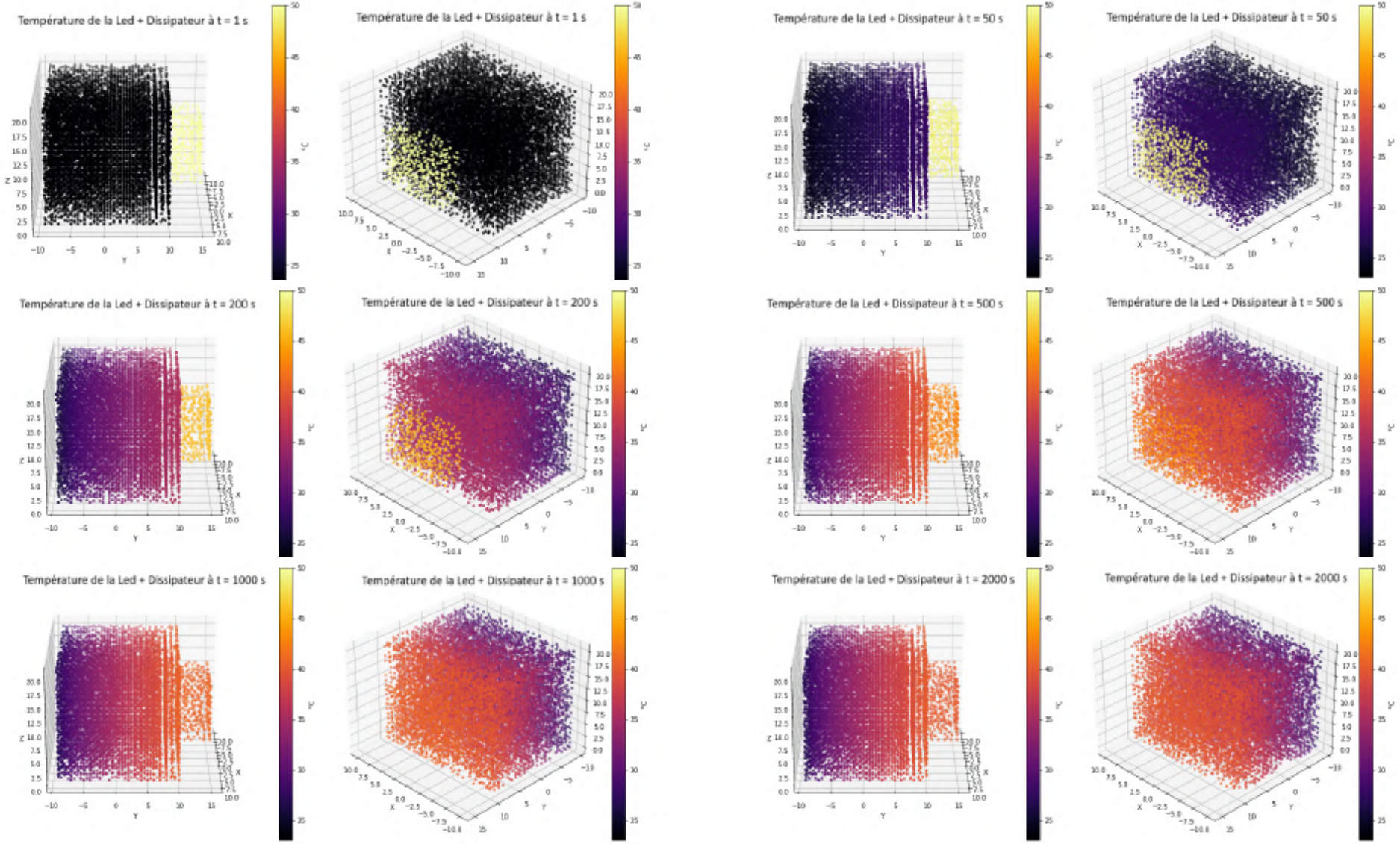


Figure: Température ambiante 23.1 °C

Simulation


Température de la Led + Dissipateur à t = 1 s
Température de la Led + Dissipateur à t = 1 s
Température de la Led + Dissipateur à t = 50 s
Température de la Led + Dissipateur à t = 50 s
Température de la Led + Dissipateur à t = 200 s
Température de la Led + Dissipateur à t = 200 s
Température de la Led + Dissipateur à t = 500 s
Température de la Led + Dissipateur à t = 500 s
Température de la Led + Dissipateur à t = 1000 s
Température de la Led + Dissipateur à t = 1000 s
Température de la Led + Dissipateur à t = 2000 s
Température de la Led + Dissipateur à t = 2000 s

Simulation

Sans dissipateur

- Température maximale : **58°C** au bout de **950s** = **15.8min**.
- Variations de température très faibles après **950s**.
  Régime permanent atteint.

Avec dissipateur

- Température de la LED qui décroît de **50°C** à **38.7°C**.
- Variations de température très faibles après **900s** = **15min**.
  Régime permanent atteint.

Le dissipateur permet une réduction de température de **11.3°C**, *améliorant significativement la dissipation thermique.*

Comparaison

LED sans dissipateur	**Manipulation**	**Simulation**
Temps à 5% (tr5%)	1356 s	700 s
Température moyenne finale de la LED	49 °C	58 °C

LED avec dissipateur	**Manipulation**	**Simulation**
Temps à 5% (tr5%)	708.1 s	750 s
Température moyenne finale de la LED	35 °C	38.7 °C

Comparaison des temps et températures pour les conditions avec et sans dissipateur, selon manipulation et simulation.

Comparaison

Base du Dissipateur	Manipulation	Théorie
Temps à 5% (tr5%)	899.1 s	698.5 s
Température finale	34.2 °C	47.97 °C

Bout d'ailette	Manipulation	Théorie
Temps à 5% (tr5%)	825.2 s	699.5 s
Température finale	32 °C	47.73 °C

Comparaison des temps et températures à la base et au bout de l'ailette du dissipateur selon manipulation et théorie.

Comparaison

- **Manipulation**
  - Erreurs de mesure
  - Bruit, convection naturelle
  - Défauts matériaux
- **Simulation**
  - Maillage discret
  - Conditions limites approximatives
- **Théorie**
  - Matériau homogène
  - Géométrie parfaite
- **Sources d'écarts**
  - Simplifications
  - Imprécisions paramètres
  - Incertitudes expérimentales
  - Phénomènes non modélisés

Conclusion

Malgré les simplifications faites, l'optimisation des dissipateurs réduit tout de même considérablement la température des composants, améliorant ainsi leur efficacité et leur durée de vie. Toutefois, leur encombrement et leur coût restent des limites importantes.

Annexe

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

# Temps en minutes
temps_min = [0, 5, 10, 15, 20, 25, 30]

# Conversion en secondes
temps = np.array(temps_min) * 60

T_led_sans = [23.1, 32, 35, 43, 45, 48, 49]
T_led_avec = [23.1, 27, 33, 33.7, 34, 35, 35]
T_base = [23.1, 26, 31, 32.5, 33, 34, 34.2]
T_bout = [23.1, 25.2, 29.5, 30.7, 31, 31.8, 32]

u_led_sans = [0.5, 0.6, 0.6, 0.7, 0.7, 0.8, 0.8]
u_led_avec = [0.4, 0.5, 0.5, 0.54, 0.56, 0.6, 0.6]
u_base = [0.4, 0.44, 0.46, 0.48, 0.5, 0.52, 0.52]
u_bout = [0.4, 0.42, 0.44, 0.46, 0.46, 0.48, 0.48]

def calcul_tr5_interpole(temps, T, nb_points=1000):
    T_final = T[-1]
    seuil_bas = 0.95 * T_final
    seuil_haut = 1.05 * T_final
    f = interp1d(temps, T, kind='linear')
    t_fine = np.linspace(temps[0], temps[-1], nb_points)
    T_fine = f(t_fine)

    for i in range(len(t_fine)):
        if seuil_bas <= T_fine[i] <= seuil_haut:
            if np.all((T_fine[i:] >= seuil_bas) & (T_fine[i:] <= seuil_haut)):
                return t_fine[i]
    return None
```

```python
tr5_led_sans = calcul_tr5_interpole(temps, T_led_sans)
tr5_led_avec = calcul_tr5_interpole(temps, T_led_avec)
tr5_base = calcul_tr5_interpole(temps, T_base)
tr5_bout = calcul_tr5_interpole(temps, T_bout)

print(f"tr5% LED sans dissipateur : {tr5_led_sans:.1f} s")
print(f"tr5% LED avec dissipateur : {tr5_led_avec:.1f} s")
print(f"tr5% Base dissipateur : {tr5_base:.1f} s")
print(f"tr5% Bout d'ailette : {tr5_bout:.1f} s")

plt.figure(figsize=(8,5))
plt.errorbar(temps, T_led_sans, yerr=u_led_sans, fmt='o-', label='LED sans dissipateur',
             capsize=4, elinewidth=2, markeredgewidth=1, color='tab:blue')
plt.errorbar(temps, T_led_avec, yerr=u_led_avec, fmt='o-', label='LED avec dissipateur',
             capsize=4, elinewidth=2, markeredgewidth=1, color='tab:orange')
plt.errorbar(temps, T_base, yerr=u_base, fmt='s-', label='Base dissipateur (contact LED)',
             capsize=4, elinewidth=2, markeredgewidth=1, color='tab:green')
plt.errorbar(temps, T_bout, yerr=u_bout, fmt='d-', label='Bout d\'ailette dissipateur',
             capsize=4, elinewidth=2, markeredgewidth=1, color='tab:red')

plt.xlabel("Temps (s)")
plt.ylabel("Température (°C)")
plt.title("Evolution de la température : LED seule, LED + dissipateur, base et bout d'ailette")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(r"C:\Users\abdul\OneDrive\Bureau\Tipe\evolution_temperature.png", dpi=300)
plt.show()
```

Figure: Manipulation graphique

Annexe

```python
import os
import meshio
import numpy as np
import matplotlib.pyplot as plt

dossier = r"C:\Users\abdul\OneDrive\Bureau\femj0rbk1ou"
dossier_images = r"C:\Users\abdul\OneDrive\Bureau\Test"
os.makedirs(dossier_images, exist_ok=True)
time_steps = [1] + list(range(50, 2001, 50))

def tracer_temp(dossier, time_steps):
    moyennes_temp = []
    for t in time_steps:
        fichier = f"FreeCAD_t{str(t).zfill(4)}.vtu"
        chemin = os.path.join(dossier, fichier)
        if not os.path.exists(chemin):
            print(f"[!] Fichier manquant : {fichier}")
            continue
        mesh = meshio.read(chemin)
        if "temperature" not in mesh.point_data:
            print(f"[!] Pas de température dans : {fichier}")
            continue
        points = mesh.points
        temp = mesh.point_data["temperature"].flatten()
        temp_moyenne = np.mean(temp)
        moyennes_temp.append((t, temp_moyenne))
        fig = plt.figure(figsize=(14, 6))
        titre = f"Température de la Led + Dissipateur à t = {t} s"
        ax1 = fig.add_subplot(121, projection='3d')
        sc1 = ax1.scatter(points[:, 0], points[:, 1], points[:, 2], c=temp, cmap='inferno', s=10)
        ax1.set_title(titre)
        ax1.set_xlabel("X")
        ax1.set_ylabel("Y")
        ax1.set_zlabel("Z")
        ax1.view_init(elev=20, azim=0)
        plt.colorbar(sc1, ax=ax1, label="°C")
        ax2 = fig.add_subplot(122, projection='3d')
        sc2 = ax2.scatter(points[:, 0], points[:, 1], points[:, 2], c=temp, cmap='inferno', s=10)
        ax2.set_title(titre)
        ax2.set_xlabel("X")
        ax2.set_ylabel("Y")
        ax2.set_zlabel("Z")
        ax2.view_init(elev=30, azim=135)
        plt.colorbar(sc2, ax=ax2, label="°C")
        plt.tight_layout()
        nom_image = f"temp_t{str(t).zfill(4)}.png"
        chemin_image = os.path.join(dossier_images, nom_image)
        plt.savefig(chemin_image, dpi=300)
        plt.close(fig)
```

```python
    if moyennes_temp:
        t_final, temp_finale = moyennes_temp[-1]
        seuil_bas_5pct = 0.95 * temp_finale
        seuil_haut_5pct = 1.05 * temp_finale
        tr95 = None
        for t_curr, temp_curr in moyennes_temp:
            if temp_curr >= seuil_bas_5pct:
                tr95 = t_curr
                break
        tr5 = None
        for t_curr, temp_curr in moyennes_temp:
            if seuil_bas_5pct <= temp_curr <= seuil_haut_5pct:
                tr5 = t_curr
                break
        print("Résumé des moyennes de température :")
        for t_curr, temp_curr in moyennes_temp:
            print(f"t={t_curr}s : moyenne = {temp_curr:.2f} °C")
        if tr95 is not None:
            print(f"Temps pour atteindre 95% de la température finale (TR95%) : t = {tr95} s")
        else:
            print("95% de la température finale non atteint dans les temps simulés.")
        if tr5 is not None:
            print(f"Temps de réponse à ±5% du régime permanent (TR5%) : t = {tr5} s")
        else:
            print("Le régime permanent ±5% n'a pas été atteint dans les temps simulés.")

tracer_temp(dossier, time_steps)
```

Figure: Simulation graphique

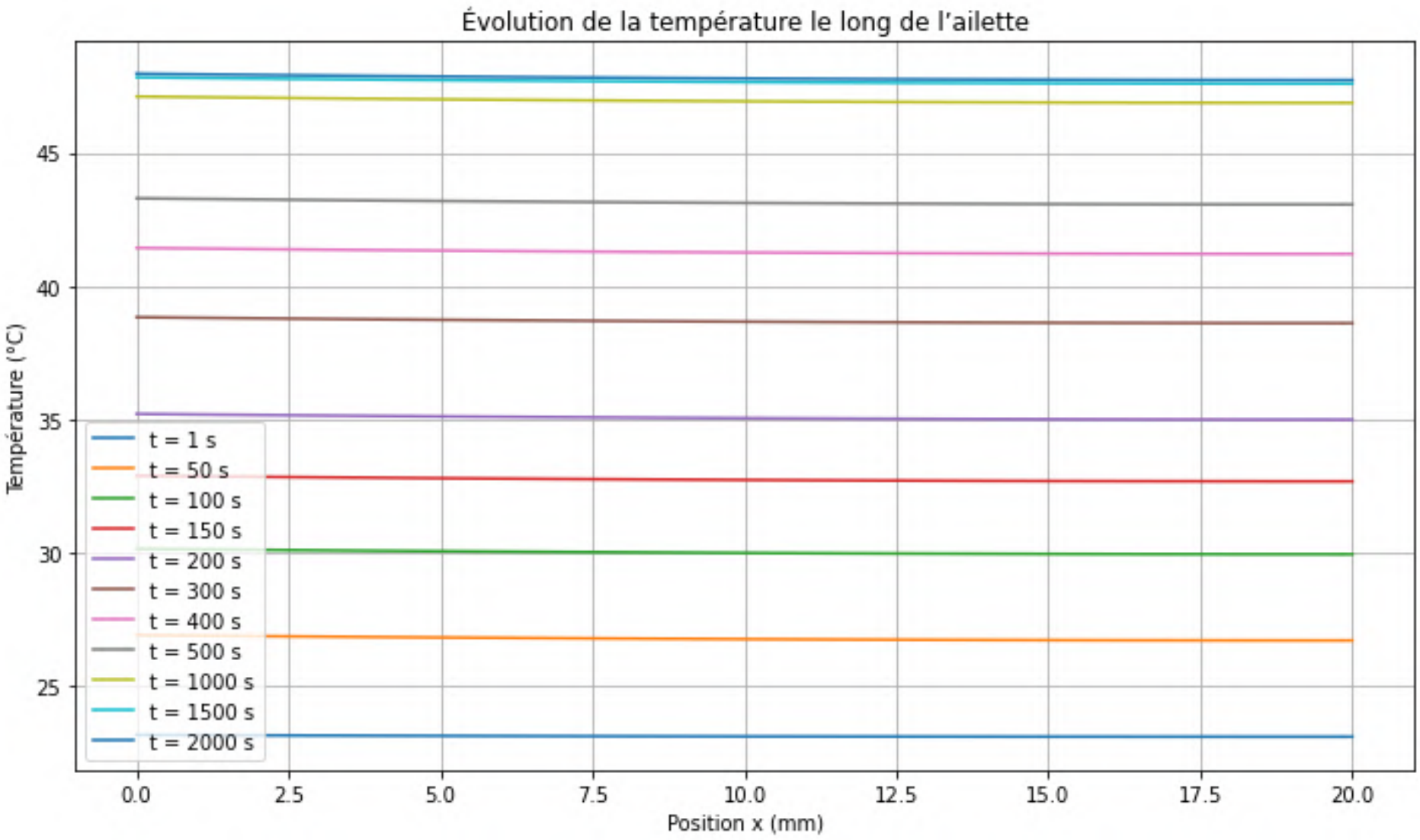
Annexe

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve

L = 20e-3
Nx = 51
dx = L / (Nx - 1)
dt = 0.5
t_max = 2000
Nt = int(t_max / dt)
rho = 2700
c = 900
lambda_ = 205
alpha = lambda_ / (rho * c)
h = 50
a = 0.01
T_inf = 23.1
beta = 2 * h / (lambda_ * a)
P_led = 10
S_contact = 0.01
q_dot = P_led / S_contact

main_diag = (1 + 2*alpha*dt/(dx**2) + alpha*dt*beta) * np.ones(Nx)
off_diag = -alpha*dt/(dx**2) * np.ones(Nx-1)
diagonals = [off_diag, main_diag, off_diag]
A = diags(diagonals, offsets=[-1, 0, 1], format='csc')
A[-1, -2] = -2 * alpha * dt / (dx**2)
A[-1, -1] = 1 + 2 * alpha * dt / (dx**2) + alpha * dt * beta

T = np.ones(Nx) * T_inf
T_base_hist = []
T_bout_hist = []
T_hist_times = [1, 50, 100, 200, 500, 1000, 1500, 2000]
T_hist = []
t_array = np.linspace(0, t_max, Nt+1)

for n in range(Nt + 1):
    t = n * dt
    S = np.zeros(Nx)
    S[0] = (q_dot * dx) / (rho * c) * dt
    b = T + alpha * dt * beta * T_inf * np.ones(Nx) + S
    T_new = spsolve(A, b)
    T = T_new.copy()
    if any(abs(t - snap) < dt/2 for snap in T_hist_times):
        T_hist.append(T.copy())
    T_base_hist.append(T[0])
    T_bout_hist.append(T[-1])
```

```python

def calc_TR5(temp_hist, t_array, T_final):
    seuil_min = 0.95 * T_final
    seuil_max = 1.05 * T_final
    for Ti, ti in zip(temp_hist, t_array):
        if seuil_min <= Ti <= seuil_max:
            return ti
    return None

T_base_finale = T_base_hist[-1]
T_bout_finale = T_bout_hist[-1]
TR5_base = calc_TR5(T_base_hist, t_array, T_base_finale)
TR5_bout = calc_TR5(T_bout_hist, t_array, T_bout_finale)

print(f"Température finale à la base (x=0) : {T_base_finale:.2f} °C")
print(f"Température finale au bout (x=L) : {T_bout_finale:.2f} °C")
print(f"TR5% à la base : {TR5_base:.1f} s" if TR5_base else "TR5% à la base : Non atteint")
print(f"TR5% au bout : {TR5_bout:.1f} s" if TR5_bout else "TR5% au bout : Non atteint")

x = np.linspace(0, L*1000, Nx)
plt.figure(figsize=(10, 6))
for idx, t_snap in enumerate(T_hist_times):
    plt.plot(x, T_hist[idx], label=f't = {t_snap} s')
plt.xlabel('Position x (mm)')
plt.ylabel('Température (°C)')
plt.title("Évolution de la température le long de l'ailette")
plt.legend(loc='upper right', fontsize=8)
plt.grid(True)
plt.tight_layout()
plt.show()
```

Figure: Théorie graphique

Annexe


Évolution de la température le long de l'ailette
Température (°C)
45
40
35
30
25
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Position x (mm)
t = 1 s
t = 50 s
t = 100 s
t = 150 s
t = 200 s
t = 300 s
t = 400 s
t = 500 s
t = 1000 s
t = 1500 s
t = 2000 s

Annexe

Calcul de l'incertitude type A (statistique) :

$$u_A = \sqrt{\frac{1}{N(N-1)} \sum_{i=1}^{N} \left(T_i - \overline{T}\right)^2}$$

avec T_i les mesures individuelles, $\overline{T}$ la moyenne, et N le nombre de mesures.

Incertitude globale u :

$$u = \sqrt{u_A^2 + u_{\text{syst}}^2}$$

où u_{syst} est l'incertitude systématique (précision, résolution, etc.)
Les incertitudes sont supposées **indépendantes**, d'où la somme quadratique.

Bibliographie

[1] IA génératrice d'images : Gemini

[2] https://northslopechillers.com/blog/heat-sinks-and-process-cooling/

[3] https://www.clubic.com/pro/entreprises/intel/actualite-373785-intel-600-les-chipsets-destines-a-alder-html

[4] https://www.amazon.fr/YUMILI-Light-Remplacement-lampoule-projecteur/dp/B09YH9GBBT

[5] https://www.servilab.fr/catalogue/produits/thermometre-checktemp-1-hanna-r

[6] https://www.servilab.fr/catalogue/produits/thermometre-checktemp-1-hanna-r

[7] https://www.freecad.org/index.php?lang=fr